

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0614

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 20%

QUESTIONS: 2

Total Marks:20

Annexure A

Industry Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

```
11. # =====
# TELECOMMUNICATION DATA PLAN RECOMMENDATION USING KNN
# Google Colab Program
# =====

# Step 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# =====
# Step 2: Create Sample Telecommunications Dataset
# =====

data = {
    "Data_Usage_GB": [
        2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
        12, 13, 15, 16, 18, 20, 22, 25, 28, 30
    ],
    "Video_Hours": [
        2, 4, 5, 7, 8, 10, 12, 14, 15, 17,
        20, 22, 25, 28, 30, 35, 38, 42, 45, 50
    ],
    "Social_Media_Hours": [
        3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
        13, 14, 15, 16, 18, 20, 22, 24, 26, 28
    ],
    "Call_Minutes": [
        100, 120, 150, 180, 200, 220, 250, 270, 300, 320,
```

```

        350, 370, 400, 420, 450, 480, 500, 550, 600, 650
    ],

    "Monthly_Spending": [
        199, 199, 249, 249, 299, 299, 349, 349, 399, 399,
        449, 449, 499, 499, 549, 599, 599, 699, 699, 799
    ],

    "Plan": [
        "Basic", "Basic", "Basic", "Basic", "Basic",
        "Standard", "Standard", "Standard", "Standard", "Standard",
        "Standard", "Standard", "Standard", "Standard",
        "Premium", "Premium", "Premium", "Premium", "Premium",
    "Premium"
    ]
}

df = pd.DataFrame(data)

print("Telecommunications Dataset")
print("=" * 50)
print(df)

# =====
# Step 3: Separate Features and Target
# =====

X = df[
    [
        "Data_Usage_GB",
        "Video_Hours",
        "Social_Media_Hours",
        "Call_Minutes",
        "Monthly_Spending"
    ]
]

y = df["Plan"]

# =====
# Step 4: Split Dataset

```

```

# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# =====
# Step 5: Feature Scaling
# =====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# =====
# Step 6: Create KNN Model
# =====

K = 5

knn = KNeighborsClassifier(n_neighbors=K)

knn.fit(X_train_scaled, y_train)

# =====
# Step 7: Predict Test Data
# =====

y_pred = knn.predict(X_test_scaled)

print("\nPredicted Plans:")
print("=" * 50)

for actual, predicted in zip(y_test, y_pred):
    print(f"Actual: {actual:10s}   Predicted: {predicted}")

```

```

# =====
# Step 8: Calculate Accuracy
# =====

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:")
print("=" * 50)
print(f"Accuracy = {accuracy * 100:.2f}%")

# =====
# Step 9: Classification Report
# =====

print("\nClassification Report:")
print("=" * 50)

print(classification_report(y_test, y_pred))

# =====
# Step 10: Confusion Matrix
# =====

cm = confusion_matrix(
    y_test,
    y_pred,
    labels=["Basic", "Standard", "Premium"]
)

plt.figure(figsize=(7, 5))

plt.imshow(cm)

plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted Plan")
plt.ylabel("Actual Plan")

plt.xticks(
    range(3),

```

```

        ["Basic", "Standard", "Premium"]
    )

plt.yticks(
    range(3),
    ["Basic", "Standard", "Premium"]
)

for i in range(3):
    for j in range(3):
        plt.text(j, i, cm[i, j],
                  ha="center",
                  va="center")

plt.colorbar()
plt.show()

# =====
# Step 11: Recommend Plan for a New Customer
# =====

print("\nEnter New Customer Usage Details")
print("=" * 50)

data_usage = float(input("Monthly Data Usage (GB): "))
video_hours = float(input("Video Streaming Hours: "))
social_hours = float(input("Social Media Hours: "))
call_minutes = float(input("Monthly Call Minutes: "))
monthly_spending = float(input("Monthly Spending (Rs): "))

new_customer = pd.DataFrame({
    "Data_Usage_GB": [data_usage],
    "Video_Hours": [video_hours],
    "Social_Media_Hours": [social_hours],
    "Call_Minutes": [call_minutes],
    "Monthly_Spending": [monthly_spending]
})

# Scale new customer's data
new_customer_scaled = scaler.transform(new_customer)

```

```

# Predict recommended plan
recommended_plan = knn.predict(new_customer_scaled)[0]

print("\nRecommended Data Plan")
print("=" * 50)
print("Recommended Plan:", recommended_plan)

# =====
# Step 12: Show Nearest Neighbours
# =====

distances, indices = knn.kneighbors(new_customer_scaled)

print("\nNearest Customers:")
print("=" * 50)

for i, (distance, index) in enumerate(
    zip(distances[0], indices[0]), start=1
):
    print(
        f"Neighbor {i}: "
        f"Customer {index + 1}, "
        f"Distance = {distance:.3f}, "
        f"Plan = {y_train.iloc[index]}"
    )

# =====
# Step 13: Recommendation Confidence
# =====

neighbor_plans = y_train.iloc[indices[0]]

plan_counts = neighbor_plans.value_counts()

confidence = (
    plan_counts[recommended_plan] / K
) * 100

print("\nRecommendation Confidence")

```

```
print("=" * 50)
print(f"{confidence:.2f}%")
```

Telecommunications Dataset

```
=====
Data_Usage_GB Video_Hours Social_Media_Hours Call_Minutes \
0      2      2      3      100
1      3      4      4      120
2      4      5      5      150
3      5      7      6      180
4      6      8      7      200
5      7     10      8      220
6      8     12      9      250
7      9     14     10      270
8     10     15     11      300
9     11     17     12      320
10     12     20     13      350
11     13     22     14      370
12     15     25     15      400
13     16     28     16      420
14     18     30     18      450
15     20     35     20      480
16     22     38     22      500
17     25     42     24      550
18     28     45     26      600
19     30     50     28      650
```

```
Monthly_Spending Plan
0      199 Basic
1      199 Basic
2      249 Basic
3      249 Basic
4      299 Basic
5      299 Standard
6      349 Standard
7      349 Standard
8      399 Standard
9      399 Standard
10     449 Standard
11     449 Standard
12     499 Standard
13     499 Standard
14     549 Premium
15     599 Premium
16     599 Premium
17     699 Premium
18     699 Premium
19     799 Premium
```

Predicted Plans:

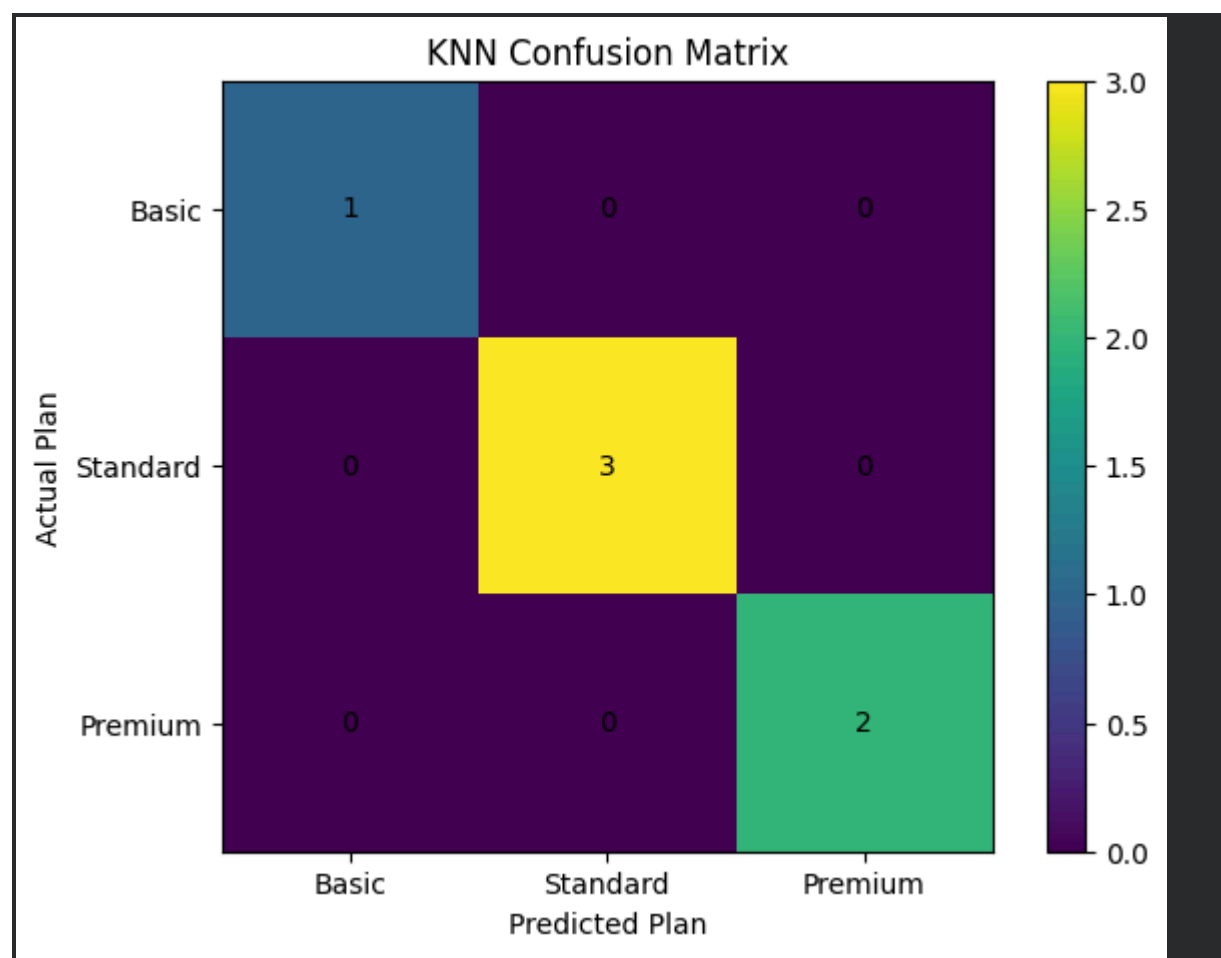
```
=====
Actual: Basic Predicted: Basic
Actual: Standard Predicted: Standard
Actual: Premium Predicted: Premium
Actual: Standard Predicted: Standard
Actual: Standard Predicted: Standard
Actual: Premium Predicted: Premium
```

Model Accuracy:

Accuracy = 100.00%

Classification Report:

	precision	recall	f1-score	support
Basic	1.00	1.00	1.00	1
Premium	1.00	1.00	1.00	2
Standard	1.00	1.00	1.00	3
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6



Enter New Customer Usage Details

Monthly Data Usage (GB): 5
Video Streaming Hours: 12
Social Media Hours: 2
Monthly Call Minutes: 5
Monthly Spending (Rs): 5

Recommended Data Plan

Recommended Plan: Basic

Nearest Customers:

```
=====
Neighbor 1: Customer 8, Distance = 1.678, Plan = Basic
Neighbor 2: Customer 7, Distance = 1.690, Plan = Basic
Neighbor 3: Customer 2, Distance = 2.025, Plan = Basic
Neighbor 4: Customer 13, Distance = 2.502, Plan = Basic
Neighbor 5: Customer 5, Distance = 2.627, Plan = Standard
```

Recommendation Confidence

```
=====
80.00%
```

```
15. # =====
# SMART CITY TRAFFIC CONGESTION PREDICTION USING KNN
# Google Colab
# =====
```

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

```
# =====
# Step 2: Create Smart City Traffic Dataset
# =====
```

```
np.random.seed(42)
```

```
n = 200
```

```
data = {
    "Vehicle_Count": np.random.randint(20, 200, n),
    "Average_Speed": np.random.randint(10, 80, n),
    "Temperature": np.random.uniform(20, 40, n),
    "Rainfall": np.random.uniform(0, 20, n),
    "Humidity": np.random.randint(40, 95, n),
    "Hour": np.random.randint(0, 24, n),
    "Previous_Traffic": np.random.randint(20, 200, n)
}
```

```
df = pd.DataFrame(data)
```

```

# =====
# Step 3: Generate Congestion Levels
# =====

def congestion_level(row):

    score = 0

    # Vehicle count
    if row["Vehicle_Count"] > 140:
        score += 2
    elif row["Vehicle_Count"] > 80:
        score += 1

    # Average speed
    if row["Average_Speed"] < 25:
        score += 2
    elif row["Average_Speed"] < 45:
        score += 1

    # Previous traffic
    if row["Previous_Traffic"] > 140:
        score += 2
    elif row["Previous_Traffic"] > 80:
        score += 1

    # Peak hours
    if row["Hour"] in [7, 8, 9, 17, 18, 19, 20]:
        score += 2

    # Rainfall
    if row["Rainfall"] > 10:
        score += 1

    if score >= 5:
        return "High"
    elif score >= 3:
        return "Medium"
    else:
        return "Low"

df["Congestion"] = df.apply(congestion_level, axis=1)

# Display dataset
print("SMART CITY TRAFFIC DATASET")
print("=" * 60)
print(df.head(10))

```

```

# =====
# Step 4: Select Features and Target
# =====

features = [
    "Vehicle_Count",
    "Average_Speed",
    "Temperature",
    "Rainfall",
    "Humidity",
    "Hour",
    "Previous_Traffic"
]

X = df[features]
y = df["Congestion"]

# =====
# Step 5: Split Dataset
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# =====
# Step 6: Feature Scaling
# =====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# =====
# Step 7: Create KNN Model
# =====

K = 5

knn = KNeighborsClassifier(n_neighbors=K)

```

```

knn.fit(X_train_scaled, y_train)

# =====
# Step 8: Predict Test Data
# =====

y_pred = knn.predict(X_test_scaled)

# =====
# Step 9: Calculate Accuracy
# =====

accuracy = accuracy_score(y_test, y_pred)

print("\nMODEL ACCURACY")
print("=" * 60)
print(f"Accuracy: {accuracy * 100:.2f}%")

# =====
# Step 10: Classification Report
# =====

print("\nCLASSIFICATION REPORT")
print("=" * 60)

print(classification_report(y_test, y_pred))

# =====
# Step 11: Confusion Matrix
# =====

labels = ["Low", "Medium", "High"]

cm = confusion_matrix(
    y_test,
    y_pred,
    labels=labels
)

plt.figure(figsize=(7, 5))

plt.imshow(cm)

plt.title("KNN Traffic Congestion - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

```

```
plt.xticks(range(3), labels)
plt.yticks(range(3), labels)
```

```
for i in range(3):
    for j in range(3):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )
```

```
plt.colorbar()
plt.show()
```

```
# =====
# Step 12: Visualize Traffic Data
# =====
```

```
plt.figure(figsize=(8, 5))
```

```
for level in ["Low", "Medium", "High"]:
```

```
    subset = df[df["Congestion"] == level]
```

```
    plt.scatter(
        subset["Vehicle_Count"],
        subset["Average_Speed"],
        label=level
    )
```

```
plt.xlabel("Vehicle Count")
plt.ylabel("Average Speed (km/h)")
plt.title("Traffic Congestion Distribution")
plt.legend()
plt.grid(True)
plt.show()
```

```
# =====
# Step 13: Real-Time Traffic Prediction
# =====
```

```
print("\nREAL-TIME TRAFFIC PREDICTION")
print("=" * 60)
```

```
vehicle_count = float(input("Vehicle Count: "))
```

```
average_speed = float(input("Average Speed (km/h): "))
temperature = float(input("Temperature (°C): "))
rainfall = float(input("Rainfall (mm): "))
humidity = float(input("Humidity (%): "))
hour = int(input("Hour (0-23): "))
previous_traffic = float(input("Previous Traffic Count: "))
```

```
# Create new traffic observation
```

```
new_data = pd.DataFrame({
    "Vehicle_Count": [vehicle_count],
    "Average_Speed": [average_speed],
    "Temperature": [temperature],
    "Rainfall": [rainfall],
    "Humidity": [humidity],
    "Hour": [hour],
    "Previous_Traffic": [previous_traffic]
})
```

```
# Scale new data
```

```
new_data_scaled = scaler.transform(new_data)
```

```
# Predict congestion
```

```
prediction = knn.predict(new_data_scaled)[0]
```

```
print("\nPREDICTION RESULT")
```

```
print("=" * 60)
```

```
print("Predicted Congestion Level:", prediction)
```

```
# =====
# Step 14: Prediction Probability
# =====
```

```
probabilities = knn.predict_proba(new_data_scaled)[0]
```

```
print("\nPrediction Probabilities:")
```

```
for label, probability in zip(knn.classes_, probabilities):
    print(f"{label}: {probability * 100:.2f}%")
```

```
# =====
# Step 15: Test Different Values of K
# =====
```

```
k_values = range(1, 16)
accuracies = []
```

```

for k in k_values:

    model = KNeighborsClassifier(n_neighbors=k)

    model.fit(X_train_scaled, y_train)

    prediction_k = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, prediction_k)

    accuracies.append(acc)

# Plot K vs Accuracy
plt.figure(figsize=(8, 5))

plt.plot(
    k_values,
    accuracies,
    marker="o"
)

plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.title("Effect of K on KNN Accuracy")
plt.xticks(k_values)
plt.grid(True)
plt.show()

# =====
# Step 16: Display Best K
# =====

best_index = np.argmax(accuracies)
best_k = list(k_values)[best_index]
best_accuracy = accuracies[best_index]

print("\nBEST K VALUE")
print("=" * 60)
print("Best K:", best_k)
print(f"Best Accuracy: {best_accuracy * 100:.2f}%")
SMART CITY TRAFFIC DATASET
=====
Vehicle_Count Average_Speed Temperature Rainfall Humidity Hour \
0      122      11  28.713458  2.677042    50  21
1      199      62  34.600786  0.273439    80  17
2      112      53  20.954323  1.507181    76  20
3       34      41  31.320744  13.834288   53  16
4      126      79  23.172929  10.686926   69   6

```

5	91	41	22.403293	14.998215	74	16
6	40	77	26.837593	18.263315	60	19
7	122	64	21.835981	11.702991	76	8
8	141	65	21.883140	14.522417	44	16
9	94	26	26.228266	15.141624	58	19

Previous_Traffic Congestion		
0	169	High
1	88	High
2	75	Medium
3	52	Low
4	55	Low
5	167	High
6	100	Medium
7	128	High
8	173	High
9	92	High

MODEL ACCURACY

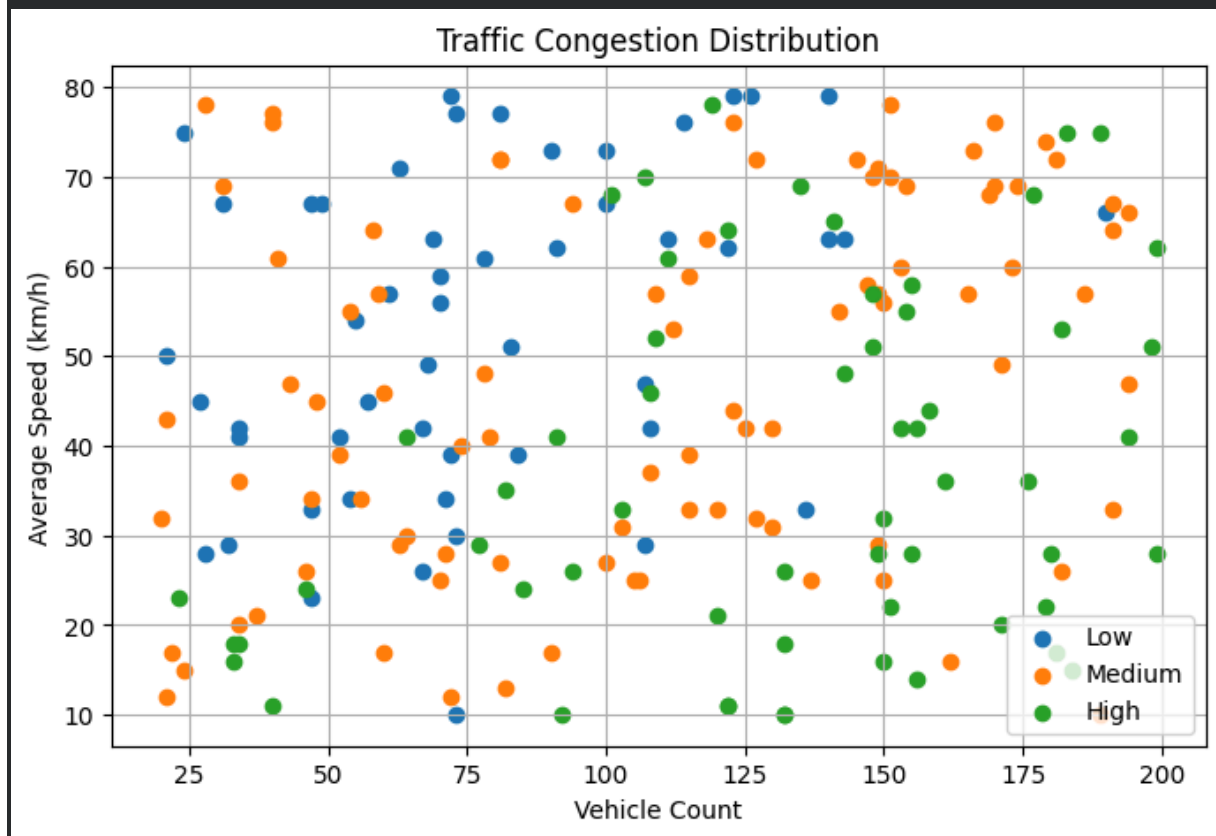
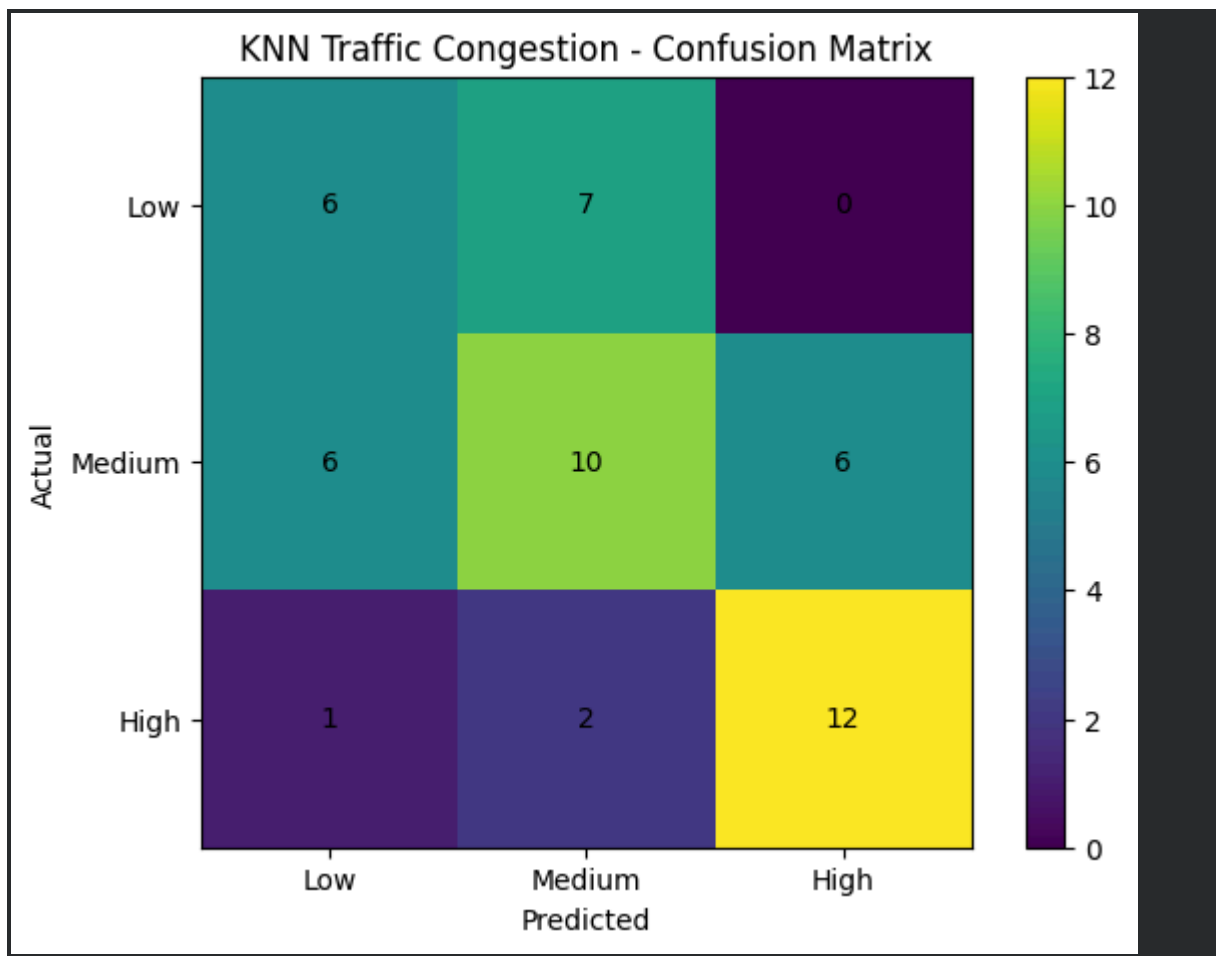
=====

Accuracy: 56.00%

CLASSIFICATION REPORT

=====

	precision	recall	f1-score	support
High	0.67	0.80	0.73	15
Low	0.46	0.46	0.46	13
Medium	0.53	0.45	0.49	22
accuracy		0.56		50
macro avg	0.55	0.57	0.56	50
weighted avg	0.55	0.56	0.55	50



REAL-TIME TRAFFIC PREDICTION

=====

Vehicle Count: 5
Average Speed (km/h): 100
Temperature (°C): 40
Rainfall (mm): 100
Humidity (%): 50
Hour (0-23): 20
Previous Traffic Count: 5

PREDICTION RESULT

=====

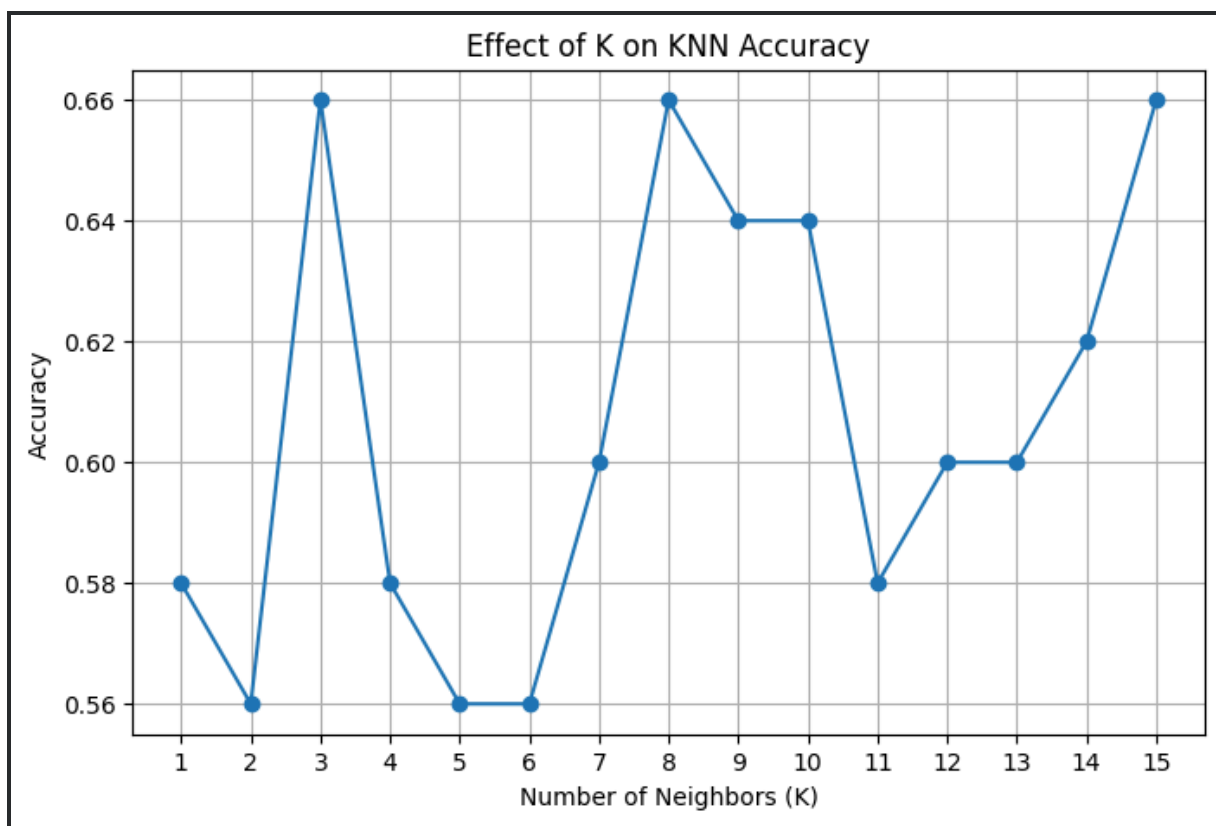
Predicted Congestion Level: Medium

Prediction Probabilities:

High: 20.00%

Low: 20.00%

Medium: 60.00%



BEST K VALUE

=====

Best K: 3

Best Accuracy: 66.00%